

CALIBRATION, PARAMETER ESTIMATION, AND TUNING

This tutorial is broken up into sections. The first section shows how to get started with the project #2 matlab materials. The sections that follow describe how to carry out the various matlab calibration procedures, parameter estimations, and PID tuning.

The various procedures described in this tutorial must be carried out in the order given. Each procedure will generally require results from previous procedures in order to operate correctly.

A. Getting Started

1. Download **utilities_robotics.zip** and place it in the **matlab/utilities** directory of the class matlab environment. Unzip this file to create the **utilities_robotics** directory. This directory contains a number of utilities to be used with the class robot. The important files are: (1) the **serial_start.m** matlab script, (2) the **serial_isrunning.m** matlab script, (3) the **get_serial_data.p** matlab function, (4) the **show_serial_data.p** matlab script, (5) the **set_speed.m** matlab function, (6) the **stop.m** matlab script, and (7) the **set_led.m** matlab function.
2. Start matlab, go to the class matlab environment root and type **init** to place all utilities in the matlab path.
3. At the class matlab environment root, download the **project2.zip** file and unzip it. This will create the **project2** directory with several matlab files. This directory contains 3 files: (1) the **runme.m** matlab script, (2) the **control_loop.p** matlab script, and (3) the **map_serial_data.m** matlab function.
4. Change the matlab directory to the **project2** directory by typing

```
>> cd project2
```

All the calibration procedures will be performed from this **project2** directory.

5. Connect the serial cable from your robot and plug it into your computer. Be sure to listen for audible feedback from your computer to ensure that your computer recognized the ATmega328PB device. This connection will cause your Windows computer to assign a *communications port* to this device. For a Windows computer, this is defined as something like **COM5** or **COM12**. It is necessary to determine which communications port *your* computer has assigned to your robot. To find this, execute the following command both *with* and *without* your robot plugged in. The difference will indicate the COM port you are looking for.

```
>> serialportlist('all')
```

Leave off the semicolon at the end of this command so the output is displayed to the matlab command window.

Once you have identified the COM port associated with your robot, the **serial_start.m** matlab script needs to be modified to set the **port** variable to the proper COM port value. Directly edit this script to set the correct port value. If you power down your computer, power back up, and plug in your robot to the serial port, the COM port value will remain the same. So once the **port** value is set in **serial_start**, you should not have to change this again.

B. Description of Functions and Scripts

When you first power up your computer to start working with your robot, you will need to initialize the matlab environment by going to the class matlab primary directory and typing the **init** command to put all the utilities in the matlab path. Once this is complete, plug in your robot to your computer, go to the **project2** directory and execute **serial_start**. This will cause your computer to start logging serial data coming from the robot.

The utilities_robotics directory:

The **serial_start** script connects matlab to the serial port and creates the **sobj** variable which has the **internal.Serialport** data type. This script must be executed before the robot will respond to commands from your computer. On occasion, it may happen that the serial port stops streaming data or that the serial port is disconnect and reconnected later. In this case, it is necessary to execute the **serial_start** script again, however, if this occurs, you *must* delete the **sobj** variable before executing **serial_start** again. This can be achieved by typing the following commands:

```
>> delete sobj
>> serial_start
```

The **serial_isrunning** script checks to make sure the serial port is streaming data. It creates/modifies the variable **serialflg**. If this variable is **true**, then the serial port is running, otherwise it will be set to **false**. This script is called by **serial_start** to make sure the serial port is running after creating the **sobj** serial port variable.

The **get_serial_data** function is used to capture the most recent block of data coming from the serial port. It can be executed by

```
>> dV = get_serial_data(sobj,class);
```

where **class** is the integer **4226** to indicate the course. The **sobj** variable is created by **serial_start**. For this course, the **dV** vector has 12 components which contain the data (in order):

```
dV = [index La Ra a0 a1 a2 a3 a4 a5 a6 a7 IRval];
```

where **index** is the ATmega328PB loop counter, **La** and **Ra** are the left and right shaft encoder angles, **a0** through **a7** are the 8 line-following sensor values, and **IRval** is the raw IR ranger value.

The **show_serial_data** matlab *function* will simply show the data read in by the **get_serial_data** function and displaying the data in the command window. To see how to use this function, simply type its name in the command window without any arguments. This function can also show calibrated data by setting the fourth argument to **true**. In this case, this function will get data from the **get_serial_data** function and pass that vector into the **map_serial_data** function described next before displaying the data.

The **set_speed** function causes the robot motors to spin, and the **stop** function causes the motors to stop. The **set_speed** function takes the form

```
>> set_speed(sobj,Lspeed,Rspeed);
```

where **sobj** is the serial port variable, **Lspeed** is the left wheel speed *index*, and **Rspeed** is the right wheel speed *index*. The speed indexes must take values in the range **[-125,125]**.

The **set_led** function causes the robot LED's to turn on/off or to flash and is intended to allow the user to provide a visual aid to help know what is going on inside their algorithm as their robot operates. The programming for the LED's is entirely up to the user. This function can be executed with the command

```
>> set_led(sobj,val,mode);
```

where **sobj** is the serial port variable, **val** is the LED on/off value, and **mode** determines if the LED's are set to *on/off* or whether they are set to *flash*. The input **val** is interpreted in binary, i.e. $b_5b_4b_3b_2b_1b_0$, so that **val** equal to, for example, $(000101)_2 = (5)_{10}$ refers to LED's 1 and 3. If **mode** is either left off or set to 0, then **val** determines which LED's are turned *on* (bit set to 1) and which are turned *off* (bit set to 0) with no LED's set to flash. If **mode** is set to 1, then **val** determines which LED's are set to flash with all other LED's left in their last on/off state.

The project2 directory:

The **control_loop** matlab *script* will be used to operate the robot in order to carry out some of the procedures described below. This matlab script will require a number of variables to be defined before running it, and upon completion it will generate a number of important variables that contain operational parameters that can be used to perform calculations. The **runme** is a matlab *script* that defines a number of variables required by **control_loop** and then executes the **control_loop** script. The **control_loop** script should always be operated by running **runme**, it should never be called directly.

The **map_serial_data** matlab *function* contains calibration operations intended to convert sensor data into calibrated data. Specifically, this function should modify the line-following sensor values to eliminate the noise floor and it should convert the raw IR sensor value into a range in units of inches. This function operates on the **dV** vector that comes from **get_serial_data** and outputs the same vector but with calibrated sensor values replacing the raw sensor values.

C. The `runme.m` Script

The `runme` script defines the following variables with brief descriptions show to the right:

```
file = 'data';           % the file name containing the saved data (*.mat)

controlflg = 0;          % 0 = don't control at all (dead-reckon)
                        % 1 = line-following control

runtime = 1;             % (sec) run time for this function

Lspeed = 0;              % nominal LEFT motor speed
Rspeed = 0;              % nominal RIGHT motor speed

prt_kk = true;           % true = print the loop index, kk
prt_rev = true;          % true = print Lrev and Rrev
prt_a = true;            % true = print line-following sensor values
prt_x = true;            % true = print line position
prt_IR = true;           % true = print IR ranger value
prt_Z = true;            % true = print the PID controller parameters, Zp,Zd,Z
prt_delay = true;        % true = print the loop delay

LLmax = 5;               % # of control loop iterations before declaring lost line
Lbreaks = 1;             % stop robot on the nth link break found
Kmin = nan;              % minimum sensor sum for presence of line
Kmax = nan;              % maximum sensor sum for pick up/stop sign

Kp = 0;                  % proportional controller constant
Ki = 0;                  % integral controller constant (NOT USED)
Kd = 0;                  % differential controller constant

class = 4226;            % DO NOT CHANGE!!!!
```

The `runme.m` script will cause these parameters to be set and then the `control_loop` script to be called. If any of these input parameters are not defined, then they are defaulted inside `control_loop` before it is executed.

The `control_loop` script will create an error variable `CLerr` upon completion. If `CLerr` is *true*, then the `control_loop` finished without error and if it is *false* then an error occurred. If an error occurs, then some indication of the error will show in the command window in red.

This script may operate differently and the parameters may be interpreted differently depending upon what other matlab functions are present at the time that `runme.m` is executed. The other matlab functions include: (1) `map_serial_data.m`, (2) `get_line_position.m`, (3) `speed_map.m`, and (4) `myPID.m`. The `get_line_position` and `myPID` functions are required for line following. The `map_serial_data` will cause the raw sensor data to get converted into calibrated data, at least if it has been completed. The `speed_map` function will cause the `Lspeed` and `Rspeed` parameters to get reinterpreted as speed in units of inches/second. If the `speed_map` function is not present, then `Lspeed` and `Rspeed` are interpreted as speed indexes that must take a value in the range $[-125, 125]$. In either case, these speed parameters determine whether and how the robot moves. If they are both zero, then the robot will not move. If one or both are non-zero, then the robot will move provided the battery is turned on and plugged into the controller board.

The `runtime`, in units of seconds, is a parameter that determines how long the control loop will operate.

The `file` parameter defines the data file name that will contain the output data from the control loop. This file will be a matlab data file so it will have the extension `".mat"`. This data file will contain a single variable `q` which will have many fields including all the input data and all the output data. The primary output data is stored in `q.dM` which is an $N \times 18$ matrix, i.e. N rows, where each row corresponds to data from one time through the control loop. The form of the k th row is:

```
q.dM(k,:) = [dV CLkk SL SR Z tnow twait];
```

where `dV` is the sensor data, possibly after getting calibrated through `map_serial_data` and where `CLkk` is the matlab control loop index, `SL` and `SR` are the current speed indexes, `Z` is the current change in turn rate, `tnow` is the matlab current time stamp in seconds, and `twait` is the control loop wait time in msec, i.e. time the loop does nothing in order to keep the loop time constant at 30msec.

The `controlflg` parameter determines whether the control loop will cause the robot to follow a line or whether it will move

without any control applied at all, or whether the robot will remain stationary. The robot will move only if either or both of **Lspeed** and **Rspeed** are non-zero, the battery is plugged in, and the battery is turned on.

The print flags, from **prt_kk** through **prt_delay** determines which parameters are displayed each time through the control loop. Depending upon what calibration or tuning procedure is being carried out, only some of these parameters might be important to display.

The parameters **LLmax**, **Lbreaks**, **Kmin** and **Kmax** are used with the line-following algorithm and are only used if **controlflg** is set to 1. The line-following algorithm is designed to follow a line and to pass by short line breaks. The number of line breaks is used in various ways to calibrate the robot parameters. A line break is typically a white patch in the line that is roughly 0.3" wide. This line break should be observable by the line-following sensors by causing the line to be lost 1 or perhaps 2 consecutive times through the loop before the line is found again. However, it is important for the robot to be able to distinguish between a short line break and the robot falling off the line completely. The parameter **LLmax** is used to distinguish between these two. It is a parameter that simply counts how many consecutive times through the control loop the line is *not* found. If the value is greater than **LLmax** then the robot stops and says the "line is lost". This parameter should be something greater than 2, say 5.

Finally, the parameter **Kmin** is used to determine if the line is lost. Each time through the loop, after capturing the line-following sensor data and eliminating the noise, the values are added together. If the sum is less than **Kmin**, then the line is declared lost for that iteration through the control loop.

The value **Kmax** is used to determine if the robot has been picked up. If the robot is picked up, then the line-following reflectance sensors will not observe any reflectance so they each will give a very large output value. In this case, the sensor sum will be a value that is much larger than the sensor sum due to the presence of a line. So if the sensor sum is greater than **Kmax**, the robot is considered picked up and the robot motors will be stopped.

Finally, the PID controller requires the controller gain parameters **Kp**, **Ki**, and **Kd**. These values will need to be calibrated so the robot follows a line smoothly, including a curved line. Even though the **control_loop** considers three controller gains, only **Kp** and **Kd** will be used to follow a line. The value for **Ki** will always be set to zero.

D. Line-Following Sensor Calibration

The line-following sensors are reflectance sensors which will return a large value when over black and a low value when over white. In principle, we want the reflectance sensor to return a zero when it is over white, however noise and other issues will generally give a non-zero value. These non-zero values will distort the line position estimation. In order to correct this, it is necessary to define a noise floor vector, **amin**, which will be used to normalize the line-following sensor values. Once the **amin** vector is defined, it can be included as part of the processing in the **map_serial_data** function. This will then be used in the control loop to provide line-following sensor normalization.

To determine the **amin** vector, place the robot in an area so the line-following sensor array is over a typical white patch. Run the **show_serial_data** function for 10 seconds with the **map_serial_data** flag set to false. It will display all the data coming from the serial port, which includes the line-following sensor array values. When the execution finishes, the mean and standard deviation of the observed data will be displayed. These values can be used to construct a suitable **amin** vector. It is important to build in a buffer to make sure that spurious noise doesn't leak through the normalization process. It is suggested that once you have the value you think you should use, add 100 to it.

Once you feel confident that the **amin** vector is suitable for acquiring good line position estimates, modify the **map_serial_data** function to include the line-following sensor normalization. Once you have modified this function, rerun **show_serial_data** but with the **map_serial_data** flag set to true. Make sure that the line-following sensor values show zero. Try allowing the robot to move slowly over a white area using the following command line

```
>> set_speed(sobj,35,35); show_serial_data(sobj,4226,2,true); stop;
```

This will operate the robot for 2 seconds with a speed index of 35. If the robot line-following sensor array stays over the white area, then look at the output data and make sure the line-following sensor array values remain at zero. If not, tweak the **amin** vector as needed.

E. The **get_line_position** Function

In order to follow a line, it is necessary to determine where the line position is located given the line-following sensor array

values. To optimize the line-position estimate, it is necessary to use the *normalize* the line following sensor array values rather than the raw sensor values. Once the **amin** vector has been found from the previous calibration procedure and the **map_serial_data.m** function modified, applying this function to a data vector acquired from **get_serial_data** will perform the necessary normalization.

Write a function with the following form

```
function x = get_line_position(aV)
```

where **aV** is the *normalized* line-following sensor array values, and where **x** is the line position in units as defined in the lecture. Take note of the fact that this function should *not* perform normalization on the **aV** values because they should have already been normalized before being put into this function. Once this function is written, the control loop will use this function to calculate the line position rather than the internal function.

Through the course of carrying out subsequent projects, it will be necessary to find the line position given all 8 normalized line-following sensor array values or in some cases *fewer* than 8 values. In particular, this function is to be written so that it provides a line position estimate for lengths of the **aV** vector of 2, 4, 6, and 8. If **aV** has fewer than 8 values, assume that the values are the *centered* values from the full array of 8 values. For example, if the length of **aV** is 2, then assume these are the values corresponding to sensor positions at -50 and 50.

When you have finished writing this function, *upload* it to your online library and *grade* it.

F. Line Threshold Parameter Estimation

Before completing this procedure, it is important that the **map_serial_data** function has been updated from the line-following calibration procedure discussed in the previous section. To follow a line, it is necessary to determine suitable values for **Kmin** and **Kmax**. This can be achieved by operating the **runme** script with the following parameters:

```
controlflg = 0;      % 0 = don't control at all (dead-reckon)
runtime = 10;        % (sec) run time for this function
lspeed = 0;          % nominal LEFT motor speed
rspeed = 0;          % nominal RIGHT motor speed
prt_kk = true;        % true = print the loop index, kk
prt_a = true;         % true = print line-following sensor values
prt_x = true;         % true = print line position
prt_delay = true;     % true = print the loop delay
```

By setting **controlflg** to zero, the PID controller will not be used. Also, setting the wheel speeds to zero will cause the robot to remain stationary.

Recall that the **control_loop** script will determine that the line has been found if the line-following sensor sum is between **Kmin** and **Kmax**. Also, the value for **Kmax** is used to determine if the robot has been picked up, which will cause the line-following sensor array values to be very large and the sum to be very large.

Place a typical line directly under the line-following sensor array and run the **runme** script. As this function displays the line-following sensor array data, move the line around the sensor array to capture variation in the captured sensor values. Take careful note of the maximum and minimum values for the sensor sum that is displayed. Use this as a guide to set values for **Kmin** and **Kmax**. Be sure to choose conservative values that have a very high chance of indicating the presence of the line, or the presence of the robot being picked up. For example, find the minimum sensor sum and then divide that value by 2 to give **Kmin**. Take the maximum sensor sum value and multiply by 2 to give **Kmax**. These values may need to be adjusted in the next calibration procedure as you attempt to tune the PID controller.

G. The **myPID.m** Function

Write the **myPID.m** function that implements a PID controller algorithm. This function is to have the form

```
function [Z,Zp,Zi,Zd] = myPID(x,Kp,Ki,Kd)
```

where **x** is the parameter to be driven to zero, **Kp** is the proportional controller gain, **Ki** is the integral controller gain, and **Kd** is the differential controller gain. The output **Z** is the total control, **Zp** is the proportional control, **Zi** is the integral control, and **Zd** is the differential control. This function should initialize all **persistent** variables if it is called with no arguments.

It should also give $z_d = 0$ the first time it is called after being initialized.

When you have finished writing this function, *upload* it to your online library and *grade* it.

H. PID Controller Calibration

If you have not yet written the `myPID.m` matlab function, then the control loop will use an *internal* `myPID` function. If you have written the `myPID` matlab function, then the control loop will use your `myPID` function to follow the line. The same will be true of the `get_line_position.m` function.

The `runme` script parameters can be set so the robot will follow a line and stop when the first line break is found. Two of these parameters requires that you have a sense of what the sum of the normalized line-following sensor sum is when the sensor is over a black line. Please a black line under the line-following sensor array and run the `runme` with the following parameters

```
controlflg = 1;      % 1 = line-following control
runtime = 3;         % (sec) run time for this function

Lspeed = 40;         % (APPROXIMATE) nominal LEFT motor speed
Rspeed = 40;         % (APPROXIMATE) nominal RIGHT motor speed

prt_kk = true;       % true = print the loop index, kk
prt_a = true;        % true = print line-following sensor values
prt_x = true;        % true = print line position
prt_z = true;        % true = print the PID controller parameters, Zp,Zd,Z
prt_delay = true;    % true = print the loop delay

LLmax = 5;          % # of control loop iterations before declaring lost line
Lbreaks = 1;        % stop robot on the nth link break found
Kmin = ??;          % user-defined value from previous calibration step
Kmax = ??;          % user-defined value from previous calibration step

Kp = ??;            % proportional controller constant
Kd = ??;            % differential controller constant
```

These parameters will cause the control loop to use the PID controller to attempt to track the line, i.e. move so the line becomes centered on the line-following sensor array.

Place the robot on the line so the line-following sensor array is *offset* from the line (not centered). Since the line is offset, the control loop will attempt to steer the robot to track the line. The overall strategy is to first determine a reasonable value for K_p , then to determine a reasonable value for K_d . Perform this calibration procedure by completing the following steps:

1. Select an initial value for K_p and set K_d to zero in the `runme` script
2. Place the robot so the line-following sensor array is **offset** from the line but is still captured by the line. Run the `runme` script and observe the behavior of the robot.
3. If the robot control is too weak, increase K_p by some scale factor and repeat step #2. If the robot control is too strong, then decrease K_p by some scale factor. Iterate steps #2 and #3 until a reasonable K_p value is found. A reasonable value should cause the robot to oscillate about the line with crossing points roughly 4" apart.
4. Select an initial value for K_d and keep K_p to be the same value from steps #2 and #3.
5. Place the robot so the line-following sensor array is **offset** from the line but is still captured by the line. Run the `runme` script and observe the behavior of the robot.
6. If the robot control is too weak, increase K_d by some scale factor and repeat step #5. If the robot control is too strong, then decrease K_d by some scale factor. Since the controller factors Z_p and Z_d are displayed, you might gain insight into how the robot is controlling by looking at these values as the robot moves forward. Iterate steps #5 and #6 until a reasonable K_d value is found. A reasonable value should cause the robot to smoothly acquire the line within about 6" of forward movement (roughly 24 control loop steps).
7. Finally, place the robot so its position on the line is at an extreme angle and make sure that the robot acquires the line smoothly and within about 30 steps through the control loop. If it does not, tweak the parameters for K_p and K_d as

needed until this is achieved. Use your intuition of how K_p and K_d affect the robot movement to complete this step. You might also consider the controller factors z_p and z_d that are displayed to gain insight into how the proportional and differential control are affecting the robot control.

This procedure is needed so the robot will drive smoothly along a line in order to carry out the next few calibration procedures. Make note of the K_p and K_d values and keep them set in the `runme` script.

I. Wheel Radius Estimation

The wheel radius R can be estimated by measuring the number of wheel revolutions for a known distance of robot travel. If this is measured while the robot is moving forward, we will consider this to achieve the *effective operational wheel radius*, R . The `control_loop.p` function can be used to achieve this. In the previous procedure, the PID controller was tuned. Therefore, it is possible to drive the robot straight by simply using the line-following algorithm to cause the robot to follow a straight line. Use a straight line track and place two line breaks, approximately 0.25" wide, on the line and measure the distance between the line breaks.

Configure the `runme` script to have the robot follow a line by setting the following parameters:

```
controlflg = 1;      % 1 = line-following control
runtime = 10;        % (sec) run time for this function

Lspeed = 40;         % (APPROXIMATE) use an index for a speed of about 6"/sec
Rspeed = 40;         % (APPROXIMATE) use the same index as Lspeed

prt_kk = true;       % true = print the loop index, kk
prt_rev = true;      % true = print Lrev and Rrev
prt_x = true;        % true = print line position
prt_time = true;     % true = print the total operational time
prt_delay = true;    % true = print the loop delay

LLmax = 5;           % # of control loop iterations before declaring lost line
Lbreaks = 2;         % stop robot on the nth link break found
Kmin = ??;           % use the value from your calibration procedure
Kmax = ??;           % use the value from your calibration procedure

Kp = ??;             % use the value from your calibration procedure
Kd = ??;             % use the value from your calibration procedure
```

These parameters will cause the robot to follow the line and, therefore, drive straight if the line is straight. When the robot reaches the first line break, the displayed parameters are reset. When the robot reaches the second line, the robot will stop and the total left/right wheel revolutions can be read off the displayed data. Average the left and right wheel revolutions to capture the average wheel revolutions. This is the average number of wheel rotations the wheel rotated through while traversing the line from the first line break to the second line break. Use these parameters to estimate the *effective operational wheel radius* R .

Operate the robot 10 times and capture 10 different estimates of the operational wheel radius, store them in a vector RV . Calculate the wheel radius *average* R_{ave} , and the wheel radius *standard deviation* R_{std} .

Once you have collected the data, create a matlab *data file* with name `data_R_cal.mat` that contains the following variables

```
RV = [...];          % vector of length 10 that contains the measured data
Rave = ???;          % effective operational radius mean
Rstd = ???;          % effective operational radius standard deviation
```

J. Wheel Base Width Estimation

The wheel base width L can be estimated by measuring the number of in-plane robot rotations of the robot relative to the number of left and right wheel rotations. The formula for this calculation is based upon the equation for ω given in the differential drive equations of motion from the lecture. If the robot is operated over a fixed time t , then the in-plane rotational angle would be $\theta = \omega t$. This relation between this and the wheel rotations is given by

$$\omega = \frac{R}{L}(\omega_R - \omega_L) \quad \Rightarrow \quad \omega t = \frac{R}{L}(\omega_R t - \omega_L t) \quad \Rightarrow \quad \theta = \frac{R}{L}(\theta_R - \theta_L)$$

The in-plane rotation is found using a large protractor and the wheel rotations are found using the wheel revolutions as measured by the shaft encoder along with the measured wheel radius R . The last equation above can be solved for L to find the wheel base width and will give the *operational wheel base width*. It is best to estimate the wheel base width with the robot moving clockwise in the plane of movement and also counterclockwise in the plane of movement.

Configure the `runme` script to have the robot follow a line by setting the following parameters:

```
controlflg = 0;      % 0 = no control algorithm used
runtime = 5;         % (sec) run time for this function

Lspeed = 40;         % left wheel speed index
Rspeed = -40;        % Rspeed = -Lspeed for stationary in-plane rotation

prt_kk = true;       % true = print the loop index, kk
prt_rev = true;       % true = print Lrev and Rrev
prt_time = true;      % true = print the total operational time
prt_delay = true;     % true = print the loop delay
```

These parameters will cause the robot to spin in place. The `Lrev` and `Rrev` displayed parameters are used to determine how many times the left and right wheels rotated. Using the known effective operational wheel radius R , these revolutions can be converted into angle in units of radians. Likewise the in-plane rotational angle can be found by counting how many times the robot makes a complete in-plane revolution and then measuring the residual in-plane rotation using a large protractor. Getting to within 10 degrees of the true angle is sufficient.

Operate the robot 10 times and capture 10 different estimates of the operational wheel base width, store them in a vector `LV`. Calculate the wheel base width *average* `Lave`, and the wheel base width *standard deviation* `Lstd`.

Once you have collected the data, create a matlab *data file* with name `data_L_cal.mat` that contains the following variables

```
LV = [...];         % vector of length 10 that contains the measured data
Lave = ???;         % effective operational wheel base width mean
Lstd = ???;         % effective operational wheel base width standard deviation
```

K. Motor Speed Calibration

The `Lspeed` and `Rspeed` values sent to the robot to cause the motors to spin are referred to as speed *indexes*, and must be in the range `[-125,125]`. These speed indexes have no inherent relationship to true rotational rate of the wheel in any physical units like radians/sec or perhaps inches/sec if the motor speeds are described in terms of forward linear movement of the wheels. The goal of this calibration procedure will be to create a function that maps from desired forward linear speed of the wheels, in units of inches/sec, into speed indexes which are then sent to the motors to cause the motors to spin. A single relationship will need to be found that is suitable for both the left and right wheels, simultaneously.

To find this relationship, measured speed in units of inches/sec must be estimated for a set of speed indexes. This can be achieved using the *control loop* to drive the robot along a straight line. If the distance traveled is known in units of inches, and the time to traverse that distance is known in units of seconds, then the average speed can easily be estimated by simple division.

Configure the `runme` script to have the robot follow a line by setting the following parameters:

```
controlflg = 1;      % 1 = line-following control
runtime = 10;        % (sec) run time for this function

Lspeed = 5;          % for each run, choose a different value
Rspeed = 5;          % for each run, always set Rspeed = Lspeed

prt_kk = true;       % true = print the loop index, kk
prt_rev = true;       % true = print Lrev and Rrev
prt_time = true;      % true = print the total operational time
prt_delay = true;     % true = print the loop delay
```

```

LLmax    = 5;           % # of control loop iterations before declaring lost line
Lbreaks   = 2;           % stop robot on the nth link break found
Kmin      = ??;          % use the value from your calibration procedure
Kmax      = ??;          % use the value from your calibration procedure

Kp = ??;                % use the value from your calibration procedure
Kd = ??;                % use the value from your calibration procedure

```

These parameters will cause the robot to follow the line and, therefore, drive straight if the line is straight. When the robot reaches the first line break, the displayed parameters will be reset, including the total execution time. When the robot reaches the second line, the robot will stop and the total travel time can be read off the displayed data.

It is possible to estimate the forward movement of the robot from the first line to the second line by using the estimate wheel radius R and the average of the left/right wheel revolutions as displayed by the output of a run. This can be used in place of the physical measurement of the distance between the first and second line break.

Operate the robot using speed indexes of

```
indexV = [5 10 15 20 25 30 35 40 45 50 55 60 65 70 75 80]
```

Each speed will require operating the control loop using the `runme` script. You will notice that several of the lowest speed indexes create so little motor torque that the robot will not move. Simply assign a zero speed to those speed indexes.

Create a plot of measured wheel speed versus `indexV` (speed index), where the horizontal axis is the speed index and the vertical axis is the measured wheel speed. You will notice that the estimated robot speed has a non-linear relationship to the speed index. Throw away the data for those speed indexes where the robot didn't move. If the desired speed is zero, then simply use a speed index of zero. The remaining data points should provide a monotonic relationship between speed index and measured speed. Use the `polyfit` function to create a curve that maps true speed into speed index. The form of this function should be

```
p = polyfit(sV,indexV,order);
```

where `sV` is the vector of estimated true speed in units of inches/sec, `indexV` is the speed vector corresponding to the `sV` vector, `order` is the polynomial order, and where `p` is the calibration vector. Choose a polynomial order that gives a reasonable model for the data.

Once the vector `p` is known, the `polyval` function can be used to map a desired speed `s` into a speed index `index` as follows:

```
index = polyval(p,s);
```

Once this data is collected, write a matlab function with the following form

```
function [Lspeed,Rspeed] = speed_map(SL,SR);
```

where `SL` is the desired left wheel speed in units of inches/sec, `SR` is the desired right wheel speed in units of inches/sec, `Lspeed` and `Rspeed` are the corresponding speed indexes. This function will need to check whether `SL` and/or `SR` are zero and set the corresponding index to zero directly, or to use the `polyval` function to perform the mapping. Hard code in the vector `p`.

Once `speed_map.m` is written, and is present in the directory where `control_loop` is found, the `Lspeed` and `Rspeed` parameters will be interpreted as *true* speed in units of inches/sec. The presence of this function changes the interpretation of the `Lspeed` and `Rspeed` variables so these must be adjusted accordingly.

Once this function is written, the control loop is designed so that the PID controller parameters should remain the same. However, it is worthwhile checking the line-following algorithm to make sure that the PID controller gains `Kp` and `Kd` are still appropriate.

Create a matlab *data file* with name `data_speed_cal.mat` that contains the calibration vectors `sV` and `indexV` as well as the calibration vector `p`.

L. IR Ranger Calibration

The IR ranger provides a value that is related to the distance to the nearest object. However, the raw value provided by the IR

ranger is not a measurement in real units, rather is it simply a number. In order to make practical use of the ranging sensor, it needs to be calibrated so that the raw measurement value can be mapped into a real distance in units of inches. In order to achieve this, it is necessary to collect data and perform a **polyfit** operation.

The **show_serial_data** function displays the data coming from the serial port and displays it in the command window. Recall that the form of this function is

```
show_serial_data(sobj,class,runtime,mapflg);
```

where **sobj** is the serial port variable, **class** is to be the class number, **runtime** is the total time to display data in units of seconds, **mapflg** determines whether the data from the serial port is passed through the **map_serial_data** function (true) or not (false). At the end of the data display, the mean and standard deviation for each column is given.

This function can be used to gather data for the purpose of creating a calibration function for the IR ranger. Set the robot on a line measure so the edge of the IR ranger sits at the origin. Place a book at a distance **D** away from the ranger and run **show_serial_data** for 2 seconds. Record the *mean* and *standard deviation* for the raw IR ranger values that result. Repeat this for the following distances

```
DV = [2:10 12:2:20 25:5:40];
```

for a total of 18 measurements. For each distance, record the raw IR ranger measurement, both *mean* and *standard deviation*. Let the raw IR ranger measurement mean vector be **IRmnV** and the raw IR ranger measurement standard deviation vector be **IRstdV**.

Plot the vector **IRmnV** versus **DV** so that **DV** is on the horizontal axis and **IRmnV** is on the vertical axis. Given this plot, determine a suitable range of true distances that can be estimated from the raw IR ranger values, call this **Dmin** and **Dmax**. Trim down the **IRmnV** and **DV** vectors to only include the data that corresponds to the **DV** values within the range from **Dmin** to **Dmax**. The trimmed down vectors **IRmnV** and **DV** should contain the same number of values.

Use the matlab **polyfit** function to find a calibration vector **IRp** that can be used to map a raw IR measurement value into a distance estimate **D** in units of inches. The form of this operation is

```
IRp = polyfit(IRmnV,DV,order);
```

where **IRmn** is the trimmed down raw IR ranger data mean vector, **DV** is the trimmed down true distance vector, and **order** is a suitable polynomial order. Choose the **order** to be sufficiently high to capture the over shape of the data curve but not so large that it varies according to each data point.

It might be beneficial to *separate the curve* into two sections and perform a **polyfit** model for each section. There are many ways to create a calibration curve that will fit to the data reasonably well.

Once the data is collected and the model is found, modify the **map_serial_data** matlab function to modify the raw IR ranger value found in the data and converts it into an estimated distance in units of inches. This modification will require that the **IRp** vector be hardcoded in.

Once the **map_serial_data** function has been modified, again run the **show_serial_data** but with the **mapflg** set to *true*. As the function operates, move a book around different distances from the IR ranger and see the displayed ranger value now represent a distance in units of inches.

Create a matlab *data file* with name **data_IR_cal.mat** that contains the calibration vectors **IRmnV**, **IRstdV**, and **DV** as well as the calibration vector **IRp**.